



ML & DL

Vishal Baraiya

Lab - 11

First Neuron

```
In [2]: import numpy as np

def step_function(x):
    if x>=0:
        return 1
    else:
        return 0

print("Setup complete")
```

Setup complete

```
In [3]: ##input
x_or = np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

##output
y_or = np.array([0,1,1,1])
```

initialize weights

```
In [4]: weights = np.array([0,0])
        bias = 0
        lr = 0.5

        print("Initial weights:",weights)
        print("Initial bias:",bias)
```

Initial weights: [0 0]
Initial bias: 0

```
In [5]: #input
        input_sample = x_or[1]
        target_value = y_or[1]

        #1. Weighted sum
        z = np.dot(input_sample , weights)+bias
        print(f"Math: ({input_sample[0]} * {weights[0]}) + ({input_sample[1]}*{weights[1]})")
        print(f"Sum(z): {z:.2f}")

        # Activation
        prediction = step_function(z)
        print("Prediction: ",prediction)

        #Error
        error = target_value - prediction
        print("Error: ",error)
```

Math: (0 * 0) + (1*0) + 0
Sum(z): 0.00
Prediction: 1
Error: 0

```
In [6]: epoch = 1
        flag = True

        while flag:
            print(f"\nEPOCH -> {epoch}")
            flag = False # Assume no errors initially

            for i in range(len(x_or)):
                x = x_or[i]
                y = y_or[i]

                # Predict
                z = np.dot(x, weights) + bias
                pred = step_function(z)

                # Error
                error = y - pred

                print(f"Input: {x}, Target: {y}, Predicted: {pred}, Error: {error}")

                # Learn
                if error != 0:
                    weights = weights+lr * error * x
                    bias += lr * error
```

```

        flag = True # Error occurred, need another epoch
        print(f"Updated Weights: {weights}")
        print(f"Updated Bias: {bias}")

    epoch += 1

# Final weights and bias
print("\nTraining complete.")
print(f"Final Weights: {weights}")
print(f"Final Bias: {bias}")

```

```

EPOCH -> 1
Input: [0 0], Target: 0, Predicted: 1, Error: -1
Updated Weights: [0. 0.]
Updated Bias: -0.5
Input: [0 1], Target: 1, Predicted: 0, Error: 1
Updated Weights: [0. 0.5]
Updated Bias: 0.0
Input: [1 0], Target: 1, Predicted: 1, Error: 0
Input: [1 1], Target: 1, Predicted: 1, Error: 0

```

```

EPOCH -> 2
Input: [0 0], Target: 0, Predicted: 1, Error: -1
Updated Weights: [0. 0.5]
Updated Bias: -0.5
Input: [0 1], Target: 1, Predicted: 1, Error: 0
Input: [1 0], Target: 1, Predicted: 0, Error: 1
Updated Weights: [0.5 0.5]
Updated Bias: 0.0
Input: [1 1], Target: 1, Predicted: 1, Error: 0

```

```

EPOCH -> 3
Input: [0 0], Target: 0, Predicted: 1, Error: -1
Updated Weights: [0.5 0.5]
Updated Bias: -0.5
Input: [0 1], Target: 1, Predicted: 1, Error: 0
Input: [1 0], Target: 1, Predicted: 1, Error: 0
Input: [1 1], Target: 1, Predicted: 1, Error: 0

```

```

EPOCH -> 4
Input: [0 0], Target: 0, Predicted: 0, Error: 0
Input: [0 1], Target: 1, Predicted: 1, Error: 0
Input: [1 0], Target: 1, Predicted: 1, Error: 0
Input: [1 1], Target: 1, Predicted: 1, Error: 0

```

```

Training complete.
Final Weights: [0.5 0.5]
Final Bias: -0.5

```

In []:

In []: *## perform same as for AND gate*